

Data Structure - is a way of organising data so that it can be used efficiently.

Why data structures?

- i) They are essential ingredients in creating fast and powerful algorithms.
- ii) They help to manage and organise data.
- iii) They make code cleaner and easier to understand.

Abstract Data Type (ADT) - Some mode of transportation from point A to point B exactly is Data structure. that is Abstract data type & how did we do that

Abstraction (ADT)	Implementation (DS)
List	Dynamic Array Linked List
Queue	Linked List based Queue Array based Queue Stack based Queue
Map	Tree Map Hash Map / Hash Table
Vehicle	Golf Cart Bicycle Smart Car

ADT defines how a data structure should behave and what methods it should have but not the details surrounding how these methods are implemented.

Complexity Analysis -

- i) **Big-O Notation** - Worst case in time and space.

n - The size of the input.

Constant Time : $O(1)$

Logarithmic Time : $O(\log(n))$

Linear Time : $O(n)$

Linearithmic Time : $O(n \log(n))$

Quadratic Time : $O(n^2)$

Cubic Time : $O(n^3)$

Exponential Time : $O(b^n)$, $b > 1$.

Factorial Time : $O(n!)$

Big-O Properties -

$$O(n+c) = O(n)$$

$$O(cn) = O(n), \quad c > 0$$

Constant added or multiplied to n is not really a difference maker

Since, $n \gg c$.

i) The following run in constant time: $O(1)$

```
a := 1
b := 2
c := a + 5 * b
```

```
i := 0
While i < 11 Do
  i = i + 1
```

ii) The following run in linear time: $O(n)$

```
i := 0
While i < n Do
  i = i + 1
```

$f(n) = n$
 $O(f(n)) = O(n)$

```
i := 0
While i < n Do
  i = i + 3
```

$f(n) = n/3$
 $O(f(n)) = O(n)$

iii)

Big-O Examples

Both of the following run in quadratic time. The first may be obvious since n work done n times is $n*n = O(n^2)$, but what about the second one?

```
For (i := 0 ; i < n; i = i + 1)
  For (j := 0 ; j < n; j = j + 1)
```

$f(n) = n*n = n^2$, $O(f(n)) = O(n^2)$

```
For (i := 0 ; i < n; i = i + 1)
  For (j := i ; j < n; j = j + 1)
```

^ replaced 0 with i

11:55 / 8-02-16 - Introduction to Big O

Big-O Examples

For a moment just focus on the second loop. Since i goes from $[0, n)$ the amount of looping done is directly determined by what i is. Remark that if $i=0$, we do n work, if $i=1$, we do $n-1$ work, if $i=2$, we do $n-2$ work, etc...

So the question then becomes what is:
 $(n) + (n-1) + (n-2) + (n-3) + \dots + 3 + 2 + 1$?
Remarkably this turns out to be $n(n+1)/2$, so
 $O(n(n+1)/2) = O(n^2/2 + n/2) = O(n^2)$ ✓

Identity.

```
For (i := 0 ; i < n; i = i + 1)
  For (j := i ; j < n; j = j + 1)
```

iv)

Big-O Examples

Suppose we have a sorted array and we want to find the index of a particular value in the array, if it exists. What is the time complexity of the following algorithm?

```
low := 0
high := n-1
While low <= high Do
```

```
  mid := (low + high) / 2
```

```
  If array[mid] == value: return mid
  Else If array[mid] < value: lo = mid + 1
  Else If array[mid] > value: hi = mid - 1
```

```
return -1 // Value not found
```

Ans: $O(\log_2(n)) = O(\log(n))$

11:31 / 8-02-16 - Introduction to Big O

v)

Big-O Examples

```
i := 0
While i < n Do
  j = 0
  While j < 3*n Do
    j = j + 1
  j = 0
  While j < 2*n Do
```

Here outer loop is multiplied to the sum of inner loop.

v)

Big-0 Examples

```
i := 0
While i < n Do
  j = 0
  While j < 3*n Do
    j = j + 1
  j = 0
  While j < 2*n Do
    j = j + 1
  i = i + 1

f(n) = n * (3n + 2n) = 5n2
O(f(n)) = O(n2)
```

Here outer loop is multiplied to the sum of inner loop.

vi)

Big-0 Examples

```
i := 0
While i < 3 * n Do
  j := 10
  While j <= 50 Do
    j = j + 1
  j = 0
  While j < n*n*n Do
    j = j + 2
  i = i + 1

f(n) = 3n * (40 + n3/2) = 3n/40 + 3n4/2
O(f(n)) = O(n4)
```

Here,

→ Dominant term.

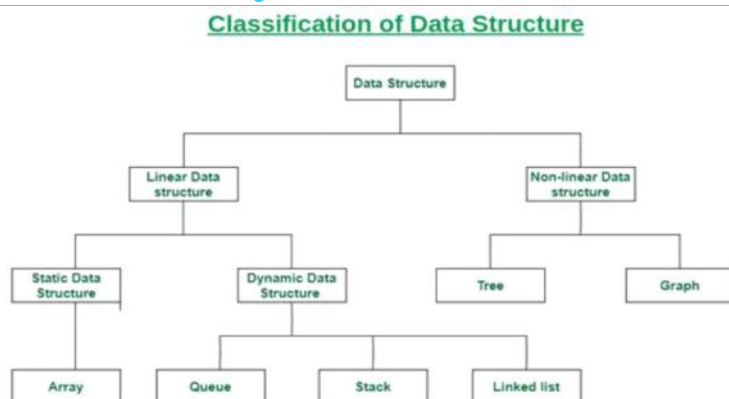
vii) Finding all subsets of a set - $O(2^n)$

Finding all permutations of a string - $O(n!)$

Sorting using mergesort - $O(n \log(n))$

Iterating over all the cells in a matrix of size n by m - $O(nm)$

Classification of Data-structures:-



Linear Data Structure:

Linear data structure: Data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure.

Examples of linear data structures are array, stack, queue, linked list, etc.

- **Static data structure:** Static data structure has a fixed memory size. It is easier to access the elements in a static data structure.

An example of this data structure is an array.

- **Dynamic data structure:** In the dynamic data structure, the size is not fixed. It can be randomly updated during the runtime which may be considered efficient concerning the memory (space) complexity of the code.

Non-linear Data Structure:

Data structures where data elements are not placed sequentially or linearly are called non-linear data structures. In a non-linear data structure, we can't traverse all the elements in a single run only.

Examples of non-linear data structures are trees and graphs.

Non-linear data structures are not easy to implement in comparison to linear data structure.

It utilizes computer memory efficiently in comparison to a linear data structure.

Difference between Linear and Non-linear Data Structures:

S.NO	Linear Data Structure	Non-linear Data Structure
1.	In a linear data structure, data elements are arranged in a linear order where each and every elements is attached to its previous and next adjacent.	In a non-linear data structure, data elements are attached in hierarchically manner.
2.	In linear data structure, single level is involved.	Whereas in non-linear data structure, multiple levels are involved.
3.	Its implementation is easy in comparison to non-linear data structure.	While its implementation is complex in comparison to linear data structure.
4.	In linear data structure, data elements can be traversed in a single run only.	While in non-linear data structure, data elements can't be traversed in a single run only.
5.	In a linear data structure, memory is not utilized in an efficient way.	While in a non-linear data structure, memory is utilized in an efficient way.
6.	Its examples are: array, stack, queue, linked list, etc.	While its examples are: trees and graphs.

Arrays form a fundamental block for all of the data structure.

Static Array - A static array is a fixed length container containing n elements indexable from the range $[0, n-1]$.

Indexable meaning that each slot/index in the array can be referenced with a number.

When & where is a static array used?

Mostly, absolutely everywhere.

i) Storing and accessing sequential data.

ii) Temporarily storing data.

iii) Used by IO routines as buffers.

iv) Look up tables and inverse lookup tables.

v) Can be used to return multiple values from a function.

vi) Used in dynamic programming to cache answers to subproblems.

Complexity		
	Static Array	Dynamic Array
Access	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$
Insertion	N/A	$O(n)$
Appending	N/A	$O(1)$
Deletion	N/A	$O(n)$

Dynamic Array:-

Dynamic Array

The dynamic array can **grow** and **shrink** in size.

A =

34	4
----	---

A.add(-7) A =

34	4	-7
----	---	----

A.add(34) A =

34	4	-7	34
----	---	----	----

A.remove(4) A =

34	-7	34
----	----	----

Dynamic Array

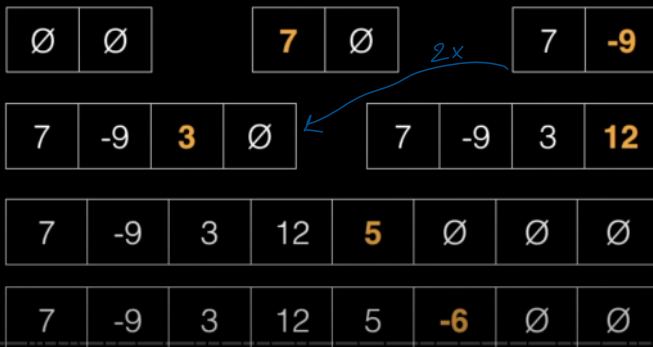
Q: How can we implement a dynamic array?

A: One way is to use a static array!

- 1) Create a static array with an initial capacity.
- 2) Add elements to the underlying static array, keeping track of the number of elements.
- 3) If adding another element will exceed the capacity, then create a new static array with twice the capacity and copy the original elements into it.

Dynamic Array

Suppose we create a dynamic array with an initial capacity of two and then begin adding elements to it.



$\phi \rightarrow$ placeholder for empty space.

Searching in Array:-

i) Sequential search.

ii) Binary search.

Sequential search-

```
#include<stdio.h>

void search(int arr[], int size, int target)
{
    int found;
    for(int i=0; i<size; i++){
        if(arr[i] == target){
            found = 1;
        }
    }
}
```

Binary search- Can only search a sorted array.

```
#include<stdio.h>

int bin(int arr[], int size, int target)
{
    int a = 0;
    int b = size - 1;
    while(a<=b){
        int mid = a+(b-a)/2;
        if(arr[mid] == target){
            return mid;
        }
    }
}
```

```

void search(int arr[], int size, int target)
{
    int found;
    for(int i=0; i<size; i++){
        if(arr[i] == target){
            found = 1;
        }
        else{
            found = 0;
        }
    }
    if(found==1){
        printf("The data is found.");
    }
    else{
        printf("The data is not found.");
    }
}

int main()
{
    int size, target;
    printf("Enter the size of your array: ");
    scanf("%d", &size);
    int arr[size];
    printf("Enter the elements of your array: ");
    for(int i=0; i<size; i++){
        scanf("%d", &arr[i]);
    }
    printf("Enter the element to search for: ");
    scanf("%d", &target);
    search(arr, size, target);
    return 0;
}

```

```

int bin(int arr[], int size, int target)
{
    int a = 0;
    int b = size - 1;
    while(a<=b){
        int mid = a+(b-a)/2;
        if(arr[mid] == target){
            return mid;
        }
        else if(arr[mid]<target){
            a = mid + 1;
        }
        else{
            b = mid - 1;
        }
    }
    return -1;
}

int main()
{
    int size, target;
    printf("Enter the size of your array: ");
    scanf("%d", &size);
    int arr[size];
    printf("Enter the elements of your array: ");
    for(int i=0; i<size; i++){
        scanf("%d", &arr[i]);
    }
    printf("Enter the element to search for: ");
    scanf("%d", &target);

    int index = bin(arr, size, target);
    if(index != -1){
        printf("The data is found at index %d\n", index);
    }
    else {
        printf("The data is not found\n");
    }
}

```

Types of Array:-

- i) One - dimensional array (1-D)
- ii) Two-dimensional array (2-D)
- (iii) Multi-dimensional array.

Two-dimensional arrays :-

Two Dimensional Array(2-D)

Initialization:

- Type Array_name[row_size][column_size];
- Type Array_name[row_size][column_size]={s1,s2,s3};
- Here s1,s2,s3 refers to sets of number.
- Example:
 - int A[3][3];
 - int A[3][3] = {{10,20,45},{42,79,81},{89,9,36}};

a[0][0]	a[0][1]	a[0][2]
a[1][0]	a[1][1]	a[1][2]
a[2][0]	a[2][1]	a[2][2]

arr	col[0]	col[1]	col[2]
row[0]	10	20	45
row[1]	42	79	81
row[2]	89	9	36

2D Array Arrangement

User Define:

- int A[r][n];
- Value of {r,n} given by user.

Multi-Dimensional Array:-

Multi Dimensional Array

Initialization:

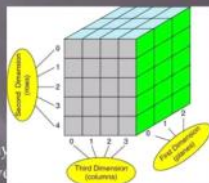
- Type Array_name[s1][s2][s3]....[sm];

Example:

- int survey [3][5][3];
- float table [5][4][5][3];

Pros & Cons:

- Two dimensional array often consider as multi dimensional array.
- Multi dimensional array is derived from basic C Language.



Multi Dimensional array Example

Insertion in Array -

Insertion In an Array

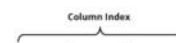
One Dimensional Array(1-D):

- Loops runs from 0 to n-1 th index.
- Sorting might involve if data was ask in sorting mode.

Insertion In An Array

Two Dimensional Array(2 - D):

- Loops runs



Insertion in an Array

One Dimensional Array(1-D):

- Loops runs from 0 to n-1 th index.
- Sorting might involve if data was ask in sorting mode.

How 1-D array insertion Works:

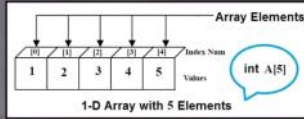
For 0 to n-1

[input value;(Using scanf)]

int A[5]

one dimension

1	A[0]
2	A[1]
3	A[2]
4	A[3]
5	A[4]



Insertion In An Array

Two Dimensional Array(2 - D):

- Loops runs
for 0 to row - 1
{for 0 to column - 1
input value;(scanf) }

How It works:

int A[3][4];

Step 1

		Column Index			
Row Index	0	1	2	3	4
	1				
	2				

Two-Dimensional Array

Step 2

		Column Index			
Row Index	0	1	2	3	4
	1	8	10	12	15
	2				

Two-Dimensional Array

Step 3

		Column Index			
Row Index	0	1	2	3	4
	1	8	10	12	15
	2	6	11	7	23

Two-Dimensional Array

		Column Index			
Row Index	0	A[0][0]	A[0][1]	A[0][2]	A[0][3]
	1	A[1][0]	A[1][1]	A[1][2]	A[1][3]
	2	A[2][0]	A[2][1]	A[2][2]	A[2][3]

Two-Dimensional Array

Structure

05 February 2024 12:17

- Structure name is optional when declaring a structure.

List as ADT-

- i) Store a given number of elements of a given datatype.
- ii) Write/modify element at a position.
- iii) Read element at a position.

List stores the same datatype. Are you talking about arrays?

Array is a datatype that gives us implementation for this list.

Dynamic List- i) empty list has size 0. ✓

ii) Insert. ✓✓

iii) Remove. ✓✓

iv) Count. ✓✓ (n+1 in array)

v) Read/modify element at a position. ✓✓

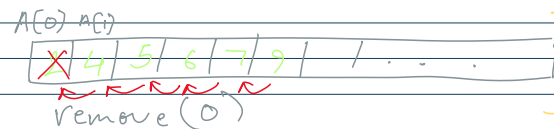
vi) Specify data-type. ✓✓

- Dynamic Arrays.



- To add 5 at index [2]
Shift 1 unit to right.

```
int A[MAX_SIZE];
int end = -1;
insert(2);
insert(4);
insert(6);
insert(7);
insert(9);
insert(5, 2);
```



To remove 2
at index [0]
Shift 1 unit
to left.

» For a dynamic array, the array size will be of some Max size. But what is a good max size? There is no good max size, there will come a time when we exhaust the whole array. To extend the currently full array we will have to create a new larger array and copy all the elements from current array to that array and then we free the memory of previous array.

This whole operation of creating a new array and copying all the elements is costly in terms of time. The (strategy) is to create a new array with double the size of currently exhausted array.

Time complexity of operations in array-

i) Access- Read/Write element at an index

Constant time

(In array elements are arranged in contiguous blocks.)

$O(1)$

standing address or base address of the block of memory and index and position of the element, we can calculate the address of that particular element and access it in constant time).

ii) Insert- length of the list (n).

$O(n)$.

...

...

ii) Remove - length/size of the list (n) $\underline{\quad O(n)}$

iii) Remove - length/size of the list (n) $\underline{\quad O(n)}$

iv) Adding (if the list isn't full) $\underline{\quad \text{constant time, } O(1)}$

Adding when the list is full $\underline{\quad O(n)}$

★ Dynamic Array isn't efficient in terms of memory.

★ Linked list is efficient in dynamic aspect in terms of memory.

Linked List:-

Albert

```
int x;  
x = 8;  
int A[4];  
A[3] = 2; // constant time  
          ↓  
          201 + 3 × 4 = 213
```

int = 4 bytes

Array is saved as block of memory

Memory

Memory Manager

Albert

```
int x;  
x = 8;  
int A[4];
```

To extend the array.

Memory

Memory Manager

8 on x is a hurdle so, we create a new array with greater size and copy the elements there.

Introduction to linked list

Memory

Memory Manager

Although it does the job, but potentially extra space may be wasted.

Introduction to linked list

Stored individually, but there is no connection/link b/w the elements.

Memory

Albert
6, 5, 4, 2

Can be linked by reserving extra memory next to the stored data. The extra memory has the address of next element. This pair is called node.

Memory Manager

Introduction to linked list

Memory

Albert
6, 5, 4, 2

```

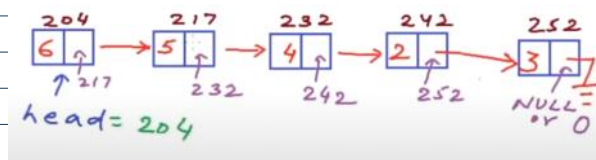
Struct Node
{
    int data; // 4 bytes
    Node* next; // 4 bytes
}
    
```

pointer var. to store address of next node.

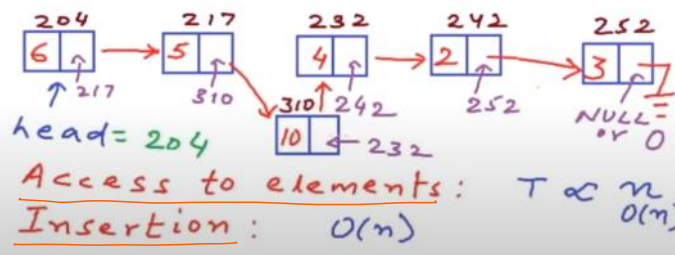
head node - address of the head node gives us access of the complete list

To access an element in linked list - We first go to the head node and ask for the address of next node and so on and so on.

Insert a node -



Create a node anywhere in the list and adjust the node properly.



* Inserting elements is a less tedious process as you don't need to shift the element to the left or right.

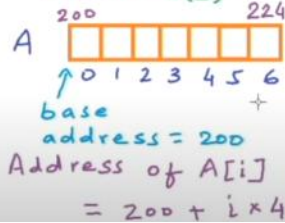
Array v/s Linked list-

Array vs Linned List

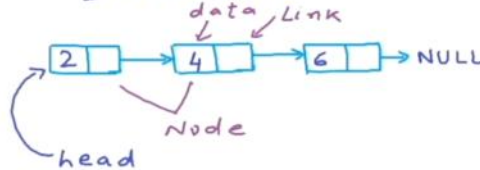
Array

1) Cost of accessing an element

Constant time - $O(1)$



Linned List



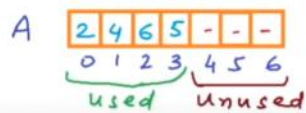
Average case: $O(n)$

Array vs Linned List

Array

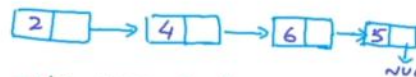
2) Memory requirements

- Fixed size



- memory may not be available as one large block

Linned List

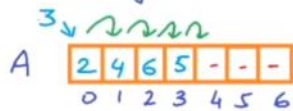


- NO unused memory
- extra memory for pointer variables

- memory may be available as multiple small blocks

Array

3) Cost of inserting an element

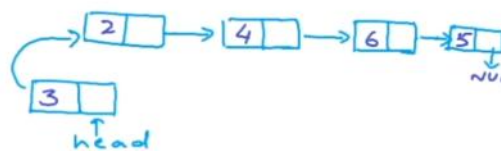


a) at beginning - $O(n)$

b) at end - $O(1)$

c) at i^{th} position - $O(n)$

Linned List



$O(1)$

$O(n)$

$O(n)$

Array vs Linned List

Array

Linned List

4) Ease of use

✓

✗

★ Linked list is prone to more errors (segmentation fault, memory leak etc.)

Implementation of Linked list -

» The address of linked list need not be in any order, it can be random.

Pointer to node - stores the address of the first node (head node). Can also be interpreted as the name for the linked list.

Struct Node
{

